Desarrollo de un sistema de seguridad VPN con un agente de inteligencia artificial para detectar actividades anómalas

Juan Sebastian Rodriguez Carreño
Universidad Distrital Francisco José de Caldas
Mecanismos de seguridad informática

***Abstract*—This paper presents a small-scale remote-access VPN prototype based on WireGuard and a local traffic-analysis agent. The server authorizes peers by their public keys, while the agent observes packet metadata on the decrypted tunnel interface and groups it into fixed windows. It does not store packet captures or inspect application payloads. A supervised Random Forest classifier and explicit rules based on destination ports and traffic volume identify windows for review. The prototype also applies an experimental DSCP/HTB policy to server egress and generates local diagnostic reports. Tests on two Debian virtual machines produced nine independent labeled test windows; the combined classifier reached an F1 score of 1.0 on this small, controlled sample. Three QoS trials per condition enforced an aggregate rate close to 10 Mbit/s, but concurrent traffic also produced ICMP packet loss. These results describe the tested virtualized setup and do not establish performance for other VPN deployments.**

***Index Terms*—VPN, WireGuard, network security, Random Forest, anomaly detection, quality of service.**

I. Introducción

El crecimiento de las tecnologías de conectividad y la expansión del trabajo remoto han incrementado la necesidad de establecer comunicaciones seguras entre usuarios y redes privadas. En este contexto, las redes privadas virtuales (VPN) permiten crear un canal cifrado sobre una infraestructura de transporte compartida y controlar el acceso de los dispositivos autorizados. Sin embargo, la protección criptográfica del canal no determina por sí sola si el comportamiento de un dispositivo autenticado corresponde al uso esperado de la red. Por consiguiente, además de proteger la comunicación, resulta conveniente disponer de mecanismos que permitan observar los patrones de tráfico y apoyar el diagnóstico de actividades que se aparten de la operación habitual.

Esta necesidad presenta una dificultad particular cuando se trabaja con tráfico cifrado. Una captura realizada en la interfaz física del servidor muestra el transporte UDP de WireGuard, pero no expone directamente los encabezados de los paquetes internos. En cambio, la interfaz lógica del túnel permite observar dichos encabezados una vez que WireGuard ha realizado el descifrado. A partir de esta interfaz es posible obtener información como el tamaño y el tiempo de llegada de los paquetes, los protocolos utilizados y los puertos o direcciones de destino, sin almacenar las capturas completas ni inspeccionar el contenido de las aplicaciones. La elección del punto de observación, por tanto, establece tanto las características disponibles para el análisis como el alcance de la información recopilada.

En el presente trabajo se desarrolla un prototipo de acceso remoto para un escenario de pequeña oficina u hogar (SOHO), compuesto por un servidor WireGuard y un número reducido de clientes autorizados mediante sus claves públicas. Sobre el servidor se implementa un agente que recopila metadatos en la interfaz del túnel y los agrupa por dispositivo en ventanas de diez segundos. Cada ventana se representa mediante características estadísticas y contadores de tráfico, que son analizados por un clasificador supervisado Random Forest y por reglas explícitas relacionadas con la cantidad de puertos de destino y el volumen transmitido. La combinación de ambos mecanismos busca señalar comportamientos que puedan ser examinados posteriormente, sin bloquear paquetes ni retirar automáticamente la autorización de un peer.

Asimismo, el prototipo incorpora una política experimental de calidad de servicio basada en DSCP y colas HTB para clasificar ciertos flujos de salida del servidor por la interfaz VPN. El agente genera registros y reportes locales con información operativa, predicciones y métricas que permiten revisar aciertos, falsas alarmas y omisiones. De esta manera, el trabajo se orienta no solo a construir un prototipo funcional, sino también a facilitar el diagnóstico de sus resultados y de las condiciones en las que se ejecuta.

La validación se realizó en dos máquinas virtuales Debian conectadas mediante una red host-only de VirtualBox. Se estableció un túnel WireGuard entre ambas y se generó tráfico normal y un barrido acotado de puertos para capturar metadatos sobre `wg0`. Las corridas de entrenamiento y prueba se separaron por identificador, y se contrastaron tres repeticiones de tráfico TCP concurrente con y sin la política de calidad de servicio. Por el tamaño reducido de la muestra y el carácter controlado de los escenarios, las métricas se interpretan como evidencia de funcionamiento del flujo y no como una estimación general de detección. El clasificador solo puede reconocer patrones representados durante su entrenamiento; sus alertas apoyan el análisis y no reemplazan los mecanismos criptográficos de autenticación de WireGuard.

El documento presenta primero los trabajos relacionados con la clasificación y el análisis de tráfico VPN. A continuación, describe el diseño e implementación del prototipo y el procedimiento empleado para evaluar sus componentes. Final-

mente, presenta las mediciones obtenidas en el laboratorio y deja planteadas las conclusiones para su elaboración posterior.

II. Trabajos relacionados

El análisis de tráfico cifrado mediante aprendizaje automático comprende tanto la selección de características observables como la construcción de conjuntos de datos adecuados para entrenar y evaluar los modelos. En el caso de las VPN, también es necesario definir el punto donde se capturan los paquetes, puesto que una observación tomada antes del descifrado contiene información distinta de la que puede obtenerse en la interfaz lógica del túnel. Los trabajos que se presentan a continuación aportan antecedentes para el escenario y el procedimiento de evaluación de este proyecto, aunque sus resultados no se pueden atribuir directamente a la implementación desarrollada.

II-A. Detección de ataques dentro de túneles VPN

Chidambaresh et al. [4] estudian la detección y clasificación de ataques en un entorno de laboratorio con WireGuard, con atención particular al escaneo de puertos. Su propuesta captura el tráfico del túnel, extrae características de flujo y aplica un clasificador Random Forest para distinguir inicialmente el tráfico normal del tráfico asociado a ataques; posteriormente, utiliza una segunda etapa basada en XGBoost para clasificar el tipo de ataque. Este trabajo constituye un antecedente cercano al escenario de prueba considerado aquí, ya que permite relacionar la observación del tráfico dentro del túnel con la identificación de comportamientos de escaneo. En el prototipo desarrollado se conserva el análisis binario y se combina un único Random Forest con reglas explícitas de tráfico. No se implementa la etapa XGBoost ni se reproducen las métricas reportadas por los autores.

II-B. Clasificación local y análisis por ventanas

Bucurel et al. [5] presentan un marco de análisis y clasificación de tráfico VPN orientado a dispositivos de recursos limitados, en particular Raspberry Pi, y estudian técnicas como Random Forest y máquinas de soporte vectorial. El interés de este antecedente para el proyecto está en la posibilidad de efectuar el análisis localmente y en la elección de modelos que puedan trabajar con representaciones tabulares del tráfico. La implementación actual emplea scikit-learn en un servidor Linux, por lo que no corresponde a una conversión del modelo a TensorFlow Lite ni a una evaluación realizada sobre Raspberry Pi.

Por su parte, Kotak et al. [3] analizan la clasificación de tráfico VPN cifrado mediante secuencias temporales y el modelo InceptionTime. Su estudio destaca que la evolución temporal y el orden de los paquetes pueden aportar información relevante para caracterizar el tráfico. En este prototipo se considera parte de esa dimensión temporal mediante estadísticas de tamaño e intervalos entre llegadas, calculadas dentro de ventanas de duración fija. Sin embargo, cada observación se resume en un vector de características y no se utilizan redes neuronales temporales ni se intenta clasificar aplicaciones específicas.

II-C. Etiquetado y evaluación de clasificadores

Eslami y Hamouda [1] estudian el uso de aprendizaje autosupervisado y aprendizaje confidente en clasificación de tráfico, con el propósito de abordar las dificultades asociadas a la escasez y al ruido en las etiquetas. Este antecedente es pertinente porque la evaluación supervisada del presente proyecto depende de disponer de ventanas cuyo comportamiento normal o anómalo haya sido identificado de forma confiable. La implementación no genera pseudoetiquetas ni aplica los métodos propuestos en dicho estudio; requiere datos de laboratorio etiquetados y la separación de las corridas empleadas para entrenamiento y prueba.

Sarwar et al. [2] presentan DarkDetect para la detección y categorización de tráfico de darknet, y examinan procedimientos de selección de características, balanceo de clases y clasificación profunda. Aunque la naturaleza del tráfico analizado en ese trabajo no equivale a la detección de actividad anómala en una VPN WireGuard, su evaluación por clase sirve de referencia para no limitar el análisis a la exactitud global. Por esta razón, el prototipo contempla precision, recall, F1 y matriz de confusión, además de accuracy. El modelo implementado pondera las clases durante el entrenamiento del Random Forest; no incorpora SMOTE, XGBoost ni la arquitectura CNN-LSTM descrita por esos autores.

III. Diseño e implementación

El diseño del prototipo se plantea para que sus componentes puedan ponerse en funcionamiento y diagnosticarse en un laboratorio de alcance reducido. Para ello, se considera un servidor Linux que establece el túnel WireGuard y ejecuta el agente de análisis, junto con uno o varios clientes autorizados. Cuando no se dispone de computadores adicionales, el escenario puede reproducirse mediante dos máquinas virtuales en un mismo equipo, conectadas a una red virtual aislada: una de ellas desempeña el papel de servidor y la otra genera el tráfico desde el cliente. Esta disposición permite observar los paquetes internos en el extremo servidor y mantener diferenciadas las funciones de generación y supervisión, aunque ambas máquinas compartan los recursos físicos del computador.

Desde el punto de vista funcional, el sistema se compone del túnel y su control de acceso, la captura de encabezados internos, la agregación de observaciones por dispositivo y ventana de tiempo, el análisis mediante el clasificador y las reglas, y el registro de alertas y diagnósticos. A estos elementos se suma una política experimental de calidad de servicio sobre el tráfico de salida del servidor. En las subsecciones siguientes se describen las decisiones de implementación y las condiciones necesarias para interpretar correctamente las mediciones.

III-A. Túnel y control de acceso

La interfaz WireGuard del servidor se denomina `wg0` y utiliza la dirección `10.77.0.1/24`. Los clientes autorizados reciben direcciones individuales, como `10.77.0.2/32` y, de manera opcional, `10.77.0.3/32`. En la configuración del servidor cada peer se relaciona con su clave pública y con una entrada `AllowedIPs` específica. Para este primer escenario,

las rutas del cliente se restringen al acceso a `10.77.0.1/32`, dado que se protegen los servicios del propio servidor y no se requiere reenviar tráfico hacia otras redes ni configurar NAT. El endpoint del cliente se establece con la dirección del servidor en la red del laboratorio y el puerto UDP 51820.

La generación de las claves se realiza con las utilidades `wg genkey` y `wg pubkey`; por consiguiente, el prototipo no implementa algoritmos criptográficos propios. WireGuard utiliza Curve25519 para el intercambio de claves, ChaCha20-Poly1305 para el cifrado autenticado y BLAKE2s como función hash. En el servidor, los archivos que contienen claves privadas deben conservar permisos restringidos y no se incorporan a los registros del agente. La autorización depende de la lista de peers configurados en WireGuard. El agente analiza el tráfico que llega a la interfaz del túnel, por lo que no ofrece una detección exhaustiva de intentos de autenticación que hayan sido rechazados antes de que los paquetes alcancen `wg0`.

III-B. Captura y representación de datos

La captura se realiza mediante `tshark` sobre la interfaz lógica `wg0`. De esta manera, el agente recibe los encabezados IPv4 y TCP/UDP una vez que WireGuard ha descifrado el paquete en el extremo servidor. Se registran campos como el timestamp, las direcciones, los puertos, el protocolo y la longitud del paquete. El colector no almacena archivos PCAP ni examina el payload de las aplicaciones; el análisis se restringe a los metadatos necesarios para describir el comportamiento observado.

Las observaciones se agrupan por peer en ventanas de diez segundos. En consecuencia, una muestra representa la actividad de un dispositivo durante una ventana y no una conexión TCP individual. Esta decisión permite reunir en una misma observación los intentos dirigidos a varios destinos, lo cual resulta útil para examinar un escaneo que se distribuye entre diferentes conexiones. Para el peer j y la ventana t, se considera el conjunto de paquetes $\mathcal{P}_{j,t}$ y se construye el vector $x_{j,t}$ con catorce características: cantidad total de paquetes y bytes; tasas de paquetes y bytes por segundo; media y desviación poblacional del tamaño; media y desviación de los intervalos entre llegadas; tiempo transcurrido entre el primer y el último paquete observado; cantidad de puertos y direcciones IP de destino únicos en sentido saliente; proporción de paquetes TCP salientes con SYN y sin ACK; proporción de paquetes RST; y proporción de paquetes salientes. Para el cálculo del tamaño se utiliza el campo `ip.len`.

$$\begin{aligned} x_{j,t} = [&n_p, n_b, n_p/W, n_b/W, \\ &\bar{L}, \sigma_L, \overline{\Delta t}, \sigma_{\Delta t}, \\ &T_a, N_{\text{puertos}}, N_{\text{IPs}}, \\ &r_{\text{SYN}}, r_{\text{RST}}, r_{\text{salida}}]. \end{aligned}$$

Fórmula 1. Vector de características por peer y ventana.

Los puertos y destinos únicos, así como la proporción SYN, se calculan sobre los paquetes salientes del peer; los bytes, tamaños, intervalos y paquetes RST se consideran en ambas direcciones. La dirección IP, el nombre del peer, la etiqueta, el escenario y el identificador de corrida se conservan como campos de diagnóstico, pero no se suministran al clasificador como características. Los registros que no cuentan con los campos requeridos se contabilizan como inválidos y se excluyen de los cálculos. En la captura continua también se omiten las ventanas parciales de los extremos y no se crean muestras artificiales para los periodos en que no se observó tráfico.

III-C. Clasificador y generación de alertas

El análisis supervisado se implementa mediante un pipeline de scikit-learn compuesto por `StandardScaler` y `RandomForestClassifier`. Los parámetros del escalador se calculan únicamente con las muestras destinadas al entrenamiento, de modo que la información de las ventanas de prueba no intervenga en esta etapa. El bosque se configura con cien árboles, profundidad máxima de ocho, tamaño mínimo de hoja de dos, ponderación equilibrada de las clases, una semilla fija y un hilo de ejecución. La clasificación distingue dos clases, normal y anómala, por lo que el entrenamiento requiere ejemplos etiquetados de ambas.

Para cada ventana, el modelo obtiene una puntuación $p_{j,t}$ asociada a la clase anómala y genera una alerta cuando dicha puntuación es igual o superior a 0.65. Como referencia adicional se implementan dos reglas: se activa una alerta cuando se observan al menos veinte puertos de destino únicos o cuando la tasa de tráfico supera 10^6 bytes por segundo. La decisión combinada corresponde a la disyunción entre la alerta del modelo y las alertas de las reglas. Estos umbrales se consideran valores iniciales y deben revisarse con datos independientes. Asimismo, la puntuación de Random Forest no se interpreta como una probabilidad calibrada de intrusión. Las alertas se registran para su análisis posterior y no bloquean paquetes ni revocan peers automáticamente.

III-D. Calidad de servicio y observabilidad

La configuración de calidad de servicio se aplica al tráfico IPv4 que sale del servidor por `wg0`. Mediante `iptables`, el script restablece el campo DSCP y marca como EF los paquetes ICMP y las respuestas TCP de SSH y del puerto de ensayo 5202. El flujo TCP del puerto 5201 se conserva en la clase normal. Posteriormente, un filtro de `tc` asigna los paquetes EF a una clase HTB prioritaria. Para una tasa total configurada, la clase prioritaria dispone de una garantía del 25 % y la clase normal del 75 %; ambas pueden utilizar capacidad disponible hasta alcanzar el límite del padre. La política describe únicamente la salida del servidor por la VPN, no prioriza la subida del cliente ni garantiza que otros routers conserven o apliquen las marcas DSCP. Los contadores de clases y colas se consultan como evidencia de qué tráfico fue procesado por la configuración.

Como parte de la observabilidad, el agente consulta periódicamente los metadatos que proporciona `wg`, entre ellos los peers configurados, el endpoint, la antigüedad del último handshake y los contadores de transferencia. Los estados `recent`, `stale` y `never` expresan la antigüedad de ese

handshake y, por sí solos, no demuestran la existencia o ausencia de una sesión activa. Para cada corrida se generan un manifiesto, archivos CSV de paquetes y ventanas, predicciones, errores de clasificación, alertas en formato JSONL, instantáneas de VPN y QoS, y un reporte HTML local. También se registran paquetes inválidos, tardíos o descartados de la cola, junto con el consumo de CPU y memoria del proceso Python. El retardo de inferencia corresponde al tiempo de cálculo por ventana y no incluye los diez segundos que se emplean en acumular las observaciones.

III-E. Entrenamiento y evaluación

La evaluación se realizó con capturas obtenidas por `tshark` en la interfaz `wg0` de la máquina servidor. Para la clase normal se generó tráfico ICMP y una transferencia UDP controlada hacia el puerto 5201. Para la clase anómala se ejecutó desde el cliente un barrido TCP limitado al servidor `10.77.0.1` y a los puertos 1000–1199. Cada registro de captura se etiquetó según el escenario generado; esta etiqueta describe la actividad del ensayo y no implica que se haya observado una intrusión real.

Se realizaron cuatro corridas completas: una normal y una anómala para entrenamiento, y otras dos para prueba independiente. El entrenamiento reunió 22 ventanas de diez segundos y la prueba, nueve; los identificadores de las cuatro capturas no se comparten entre conjuntos. Esta separación evita que ventanas contiguas de una misma observación aparezcan a ambos lados de la evaluación. La implementación rechaza ventanas duplicadas y duraciones incompatibles con el modelo. Debido al número limitado de corridas y al contraste marcado entre los patrones generados, las métricas se consideran exploratorias y no se extrapolan a tráfico de producción.

Tomando la clase anómala como positiva, la matriz de confusión contabiliza verdaderos positivos TP, falsos positivos FP, verdaderos negativos TN y falsos negativos FN. A partir de estos valores se calculan las métricas de evaluación:

$$\begin{aligned} \text{Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN}, \\ \text{Precision} &= \frac{TP}{TP + FP}, \\ \text{Recall} &= \frac{TP}{TP + FN}. \end{aligned}$$

Fórmula 2. Métricas de clasificación.

$$\begin{aligned} F1 &= \frac{2\,\text{Precision}\,\text{Recall}}{\text{Precision} + \text{Recall}}, \\ FPR &= \frac{FP}{FP + TN}. \end{aligned}$$

Fórmula 3. Medidas de equilibrio y falsas alarmas.

El reporte presenta por separado las métricas del modelo, de las reglas y de la decisión combinada. Si el denominador de una medida es cero, se registra como no calculable. Para la evaluación de QoS se generaron dos flujos TCP inversos de ocho Mbit/s en los puertos 5201 y 5202, junto con mediciones ICMP. Se realizaron tres corridas de veinte segundos por condición, sin QoS y con una tasa HTB de diez Mbit/s; los contadores de `tc` y las marcas DSCP se revisaron al finalizar cada repetición. Además, se midió el ping con QoS en ausencia de saturación para distinguir el comportamiento bajo carga del comportamiento en reposo.

IV. Resultados

IV-A. Túnel y capturas del agente

El laboratorio se ejecutó en dos máquinas virtuales Debian 13.7 de 1 vCPU y 1.5 GiB de memoria cada una, conectadas por la red host-only de VirtualBox. El servidor utilizó la dirección `192.168.56.10` y el cliente `192.168.56.11`; dentro de WireGuard, las direcciones fueron `10.77.0.1` y `10.77.0.2`. El servicio `wg-quick@wg0` se mantuvo activo y el registro del servidor mostró handshakes recientes y transferencia en ambos sentidos. En una verificación de cuatro solicitudes ICMP se recibieron las cuatro respuestas, sin pérdida, con un tiempo medio de 1.247 ms.

Se ejecutaron cuatro capturas independientes con ventanas de diez segundos. El conjunto de entrenamiento incluyó 11 ventanas normales, con 2,775 paquetes, y 11 ventanas del barrido, con 8,060 paquetes. La prueba independiente incluyó cinco ventanas normales con 1,231 paquetes y cuatro ventanas anómalas con 3,610 paquetes. En las ventanas del barrido se observaron 200 puertos de destino únicos; la actividad normal consistió en ping e iperf3 UDP de baja tasa. Las cuatro capturas finalizaron correctamente, sin paquetes inválidos ni descartes en la cola del agente. La Figura 1 muestra la comprobación del túnel desde la consola del cliente.

IV-B. Evaluación del detector

El modelo se entrenó con 22 ventanas reales provenientes de dos corridas y se evaluó con nueve ventanas de otras dos corridas. En la prueba se obtuvieron cinco verdaderos negativos, cuatro verdaderos positivos y ningún falso positivo o falso negativo. Random Forest, las reglas y la combinación de ambos lograron los mismos conteos en esta muestra. La Tabla I resume las métricas calculadas por el reporte.

Cuadro I
Métricas sobre nueve ventanas independientes

Detector	Acc.	Prec.	Rec.	F1	FPR
Random Forest	1.000	1.000	1.000	1.000	0.000
Reglas	1.000	1.000	1.000	1.000	0.000
Combinado	1.000	1.000	1.000	1.000	0.000

La latencia media de inferencia fue de 8.85 ms por ventana y el percentil 95 fue de 19.06 ms; este tiempo no incluye los diez segundos de acumulación de paquetes. Aunque la matriz no presenta errores, el conjunto de prueba contiene únicamente nueve ventanas y contrasta un flujo normal controlado con un barrido de 200 puertos. Por ello, el resultado no permite afirmar que se mantenga la misma exactitud ante escaneos más lentos, otros tipos de tráfico, más clientes o una red de producción.

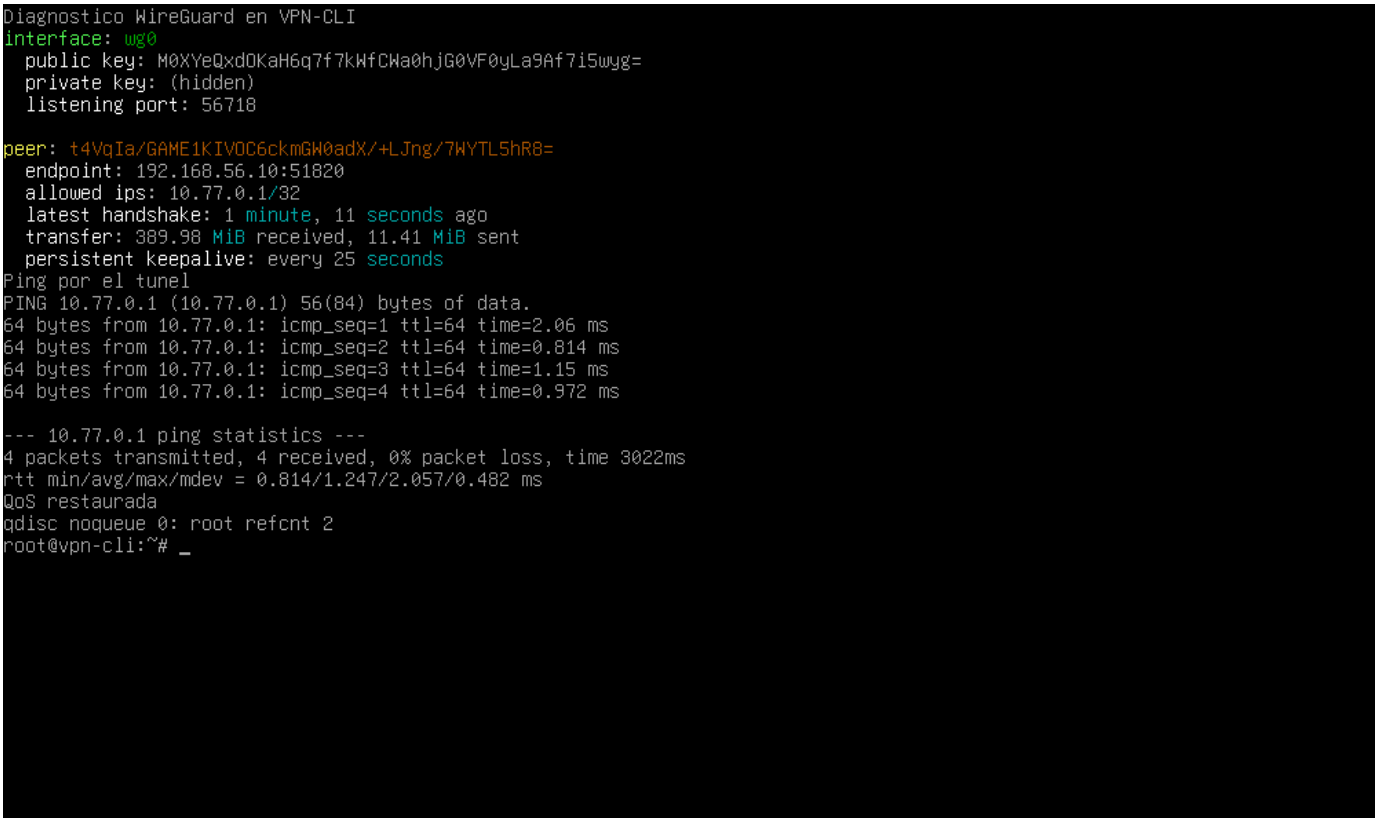


Figura 1. Handshake de WireGuard, transferencia y conectividad ICMP observados en la máquina cliente.

La Figura 2 presenta el resumen obtenido a partir del reporte de la prueba independiente.

IV-C. Diagnóstico de calidad de servicio

La política HTB se aplicó en la salida del servidor por `wg0`, con un techo de 10 Mbit/s, 2.5 Mbit/s para la clase prioritaria y 7.5 Mbit/s para la clase normal. Cada condición se repitió tres veces durante veinte segundos con dos flujos TCP inversos simultáneos, ambos solicitando 8 Mbit/s. En la condición sin QoS se midieron 8.02 Mbit/s en cada flujo. Con QoS, el flujo del puerto 5201 recibió en promedio 7.18 Mbit/s y el del puerto 5202 recibió 2.39 Mbit/s, para un total de 9.57 Mbit/s. Los contadores de `tc` registraron tráfico en las dos clases y la regla DSCP de TCP para el puerto 5202 contó paquetes, lo que confirma que la clasificación aplicada coincidió con la configuración.

La Tabla II presenta los promedios de throughput y ping bajo carga. Con QoS, la latencia ICMP media fue menor, pero se perdió en promedio el 10 % de las respuestas, con resultados entre 7 % y 14 % en las tres repeticiones. En una prueba adicional de ping sin saturación, la política recibió 100 de 100 respuestas, sin pérdida y con 1.151 ms de RTT medio. La diferencia indica que la pérdida apareció al compartir el recurso virtualizado con los dos flujos TCP; no se atribuye a una causa única. Al terminar, se retiró la política y `wg0` volvió a mostrar la cola `noqueue`.

Cuadro II
THROUGHPUT Y LATENCIA ICMP, PROMEDIO DE TRES REPETICIONES

Condición	TCP 5201 (Mbit/s)	TCP 5202 (Mbit/s)	Total (Mbit/s)	RTT medio (ms)	Pérdida ICMP (%)
Sin QoS	8.02	8.02	16.04	8.53	0.0
HTB/DSCP	7.18	2.39	9.57	6.19	10.0
HTB sin carga	–	–	–	1.15	0.0

Los registros completos de paquetes, ventanas, métricas, predicciones, errores, ping, iperf3 y contadores QoS se guardaron en ../virtualbox/resultados-vpn-20261026/. El reporte independiente del detector está disponible en ../virtualbox/resultados-vpn-20261026/evaluacion-modelo/reporte.html. Las mediciones corresponden a dos máquinas virtuales de un núcleo y no aíslan el efecto de la carga del hipervisor; además, las métricas de clasificación se basan en una muestra reducida y escenarios definidos por el propio ensayo.



Figura 2. Matriz de confusión y tiempo de inferencia del reporte del detector.

V. Conclusiones

Referencias

[1] E. Eslami and W. Hamouda, "Network Traffic Classification Using Self-Supervised Learning and Confident Learning," *IEEE Open Journal of the Communications Society*, vol. 6, pp. 9100–9120, 2025, doi: 10.1109/OJCOMS.2025.3625534.

[2] M. B. Sarwar, M. K. Hanif, R. Talib, M. Younas, and M. U. Sarwar, "DarkDetect: Darknet Traffic Detection and Categorization Using Modified Convolution-Long Short-Term Memory," *IEEE Access*, vol. 9, pp. 113705–113713, 2021, doi: 10.1109/ACCESS.2021.3105000.

[3] J. Kotak, I. Yankelev, I. Bibi, Y. Elovici, and A. Shabtai, "VPN-Encrypted Network Traffic Classification Using a Time-Series Approach," *IEEE Transactions on Network and Service Management*, vol. 22, no. 2, pp. 2225–2242, 2025, doi: 10.1109/TNSM.2025.3543903.

[4] N. Chidambaresh, S. Charukkesh, G. Dhivya, and V. N. Arumbu, "Real Time VPN Traffic Attack Detection and Classification," in *Proc. 3rd Int. Conf. Innovations in Cybersecurity and Data Science (ICICDS)*, 2026, pp. 1375–1382, doi: 10.1109/ICICDS70526.2026.11604997.

[5] M. Bucurel, S.-A. Dragusin, and N. Bizon, "Intelligent VPN Traffic Analysis and Classification Framework on Raspberry Pi Using Machine Learning Techniques," in *Proc. 18th Int. Conf. Electronics, Computers and Artificial Intelligence (ECAI)*, 2026, doi: 10.1109/ECAI69016.2026.11613590.